

EVENTHUB

Serverless Event Announcement Platform on AWS

Project Implementation, Architecture & Proof of Work

- Created by Anuj Dhirde

Project snapshot

I built a serverless event announcement website using Amazon S3, API Gateway, AWS Lambda, Amazon SNS, and IAM. Users can subscribe for notifications, create events, and view the event list dynamically through API-driven interactions.

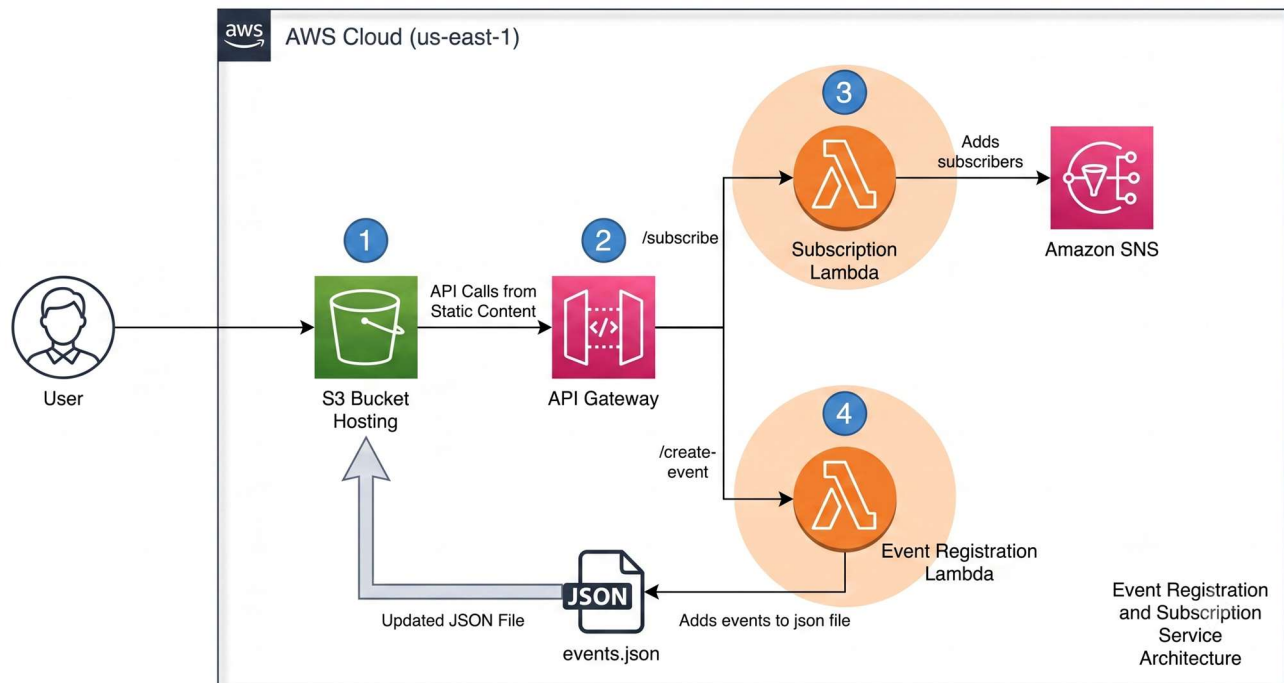


Figure 1. Original project architecture reference used for the implementation.

AWS services demonstrated: Amazon S3 • API Gateway • AWS Lambda • Amazon SNS • IAM

1. Project Overview

I built EventHub as a hands-on serverless project to demonstrate how a static web application can interact with AWS-managed APIs and serverless compute without maintaining a traditional application server. The project combines two connected workflows: subscriber registration through SNS and event management through S3-backed Lambda operations.

- Static frontend hosted on Amazon S3 using HTML, CSS, and JavaScript.
- HTTP API created with Amazon API Gateway for subscriber and event operations.
- A Subscription Lambda written in Python that adds subscriber email addresses to an SNS topic.
- An Event Registration Lambda written in Python that reads and updates events.json in S3.
- CORS configured so the S3-hosted browser application can securely call the API endpoints used by the demo.
- PowerShell and browser-based testing used to validate the end-to-end flows.

2. Implemented Architecture

I implemented the architecture in two flows. The subscription path is S3 static website → API Gateway → Subscription Lambda → SNS. The event path is the same frontend → API Gateway → Event Registration Lambda → S3 events.json. The Event Registration Lambda supports both POST for creating events and GET for returning stored events to the website.

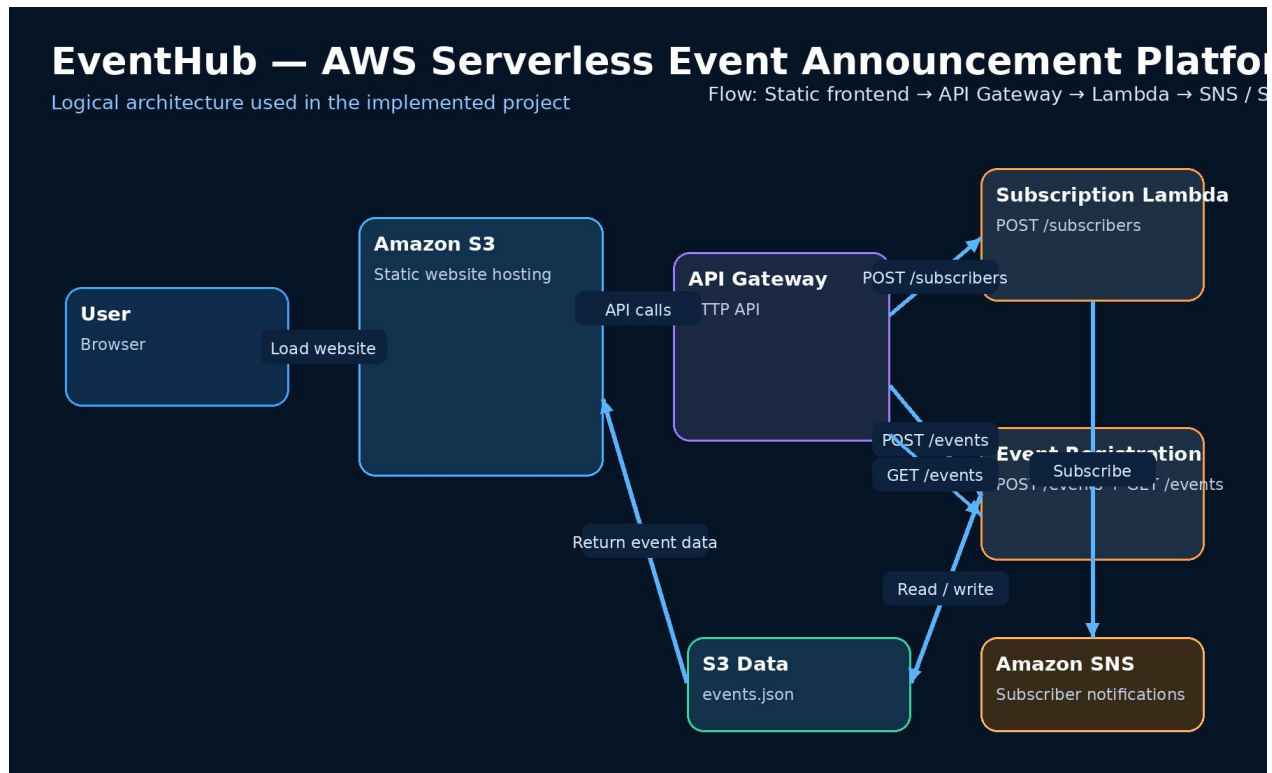


Figure 2. Logical architecture of the implemented EventHub application.

3. AWS Services I Used

Service	How I used it	Key implementation point
Amazon S3	Hosted the frontend and stored events.json.	Separated frontend hosting from backend event data.
API Gateway	Exposed HTTP endpoints for	POST /subscribers, POST /events,

	subscribers and events.	and GET /events.
AWS Lambda	Implemented subscription and event-processing logic.	Python functions with IAM execution roles.
Amazon SNS	Managed subscriber email notifications.	Subscription Lambda adds email endpoints to the topic.
IAM	Controlled Lambda access to AWS resources.	Least-privilege style permissions for SNS and S3 operations.
CORS	Allowed browser calls from the S3 website to API Gateway.	Configured the website origin plus required methods/headers.

4. Static Website Hosting on Amazon S3

I created an Amazon S3 bucket for the frontend and uploaded the three website files: index.html, style.css, and script.js. I then enabled S3 static website hosting so the application could be accessed directly from the S3 website endpoint.

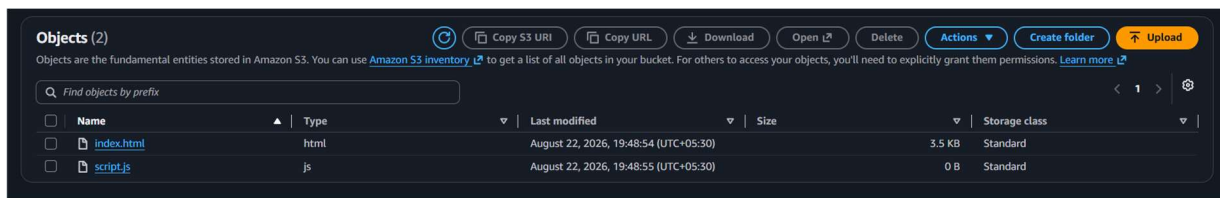


Figure 3. Frontend files uploaded to the S3 website bucket.

I configured the bucket for the static website demo and applied the required bucket policy/public-access settings for the hosted frontend. I verified the generated S3 website endpoint and opened the website successfully in a browser.

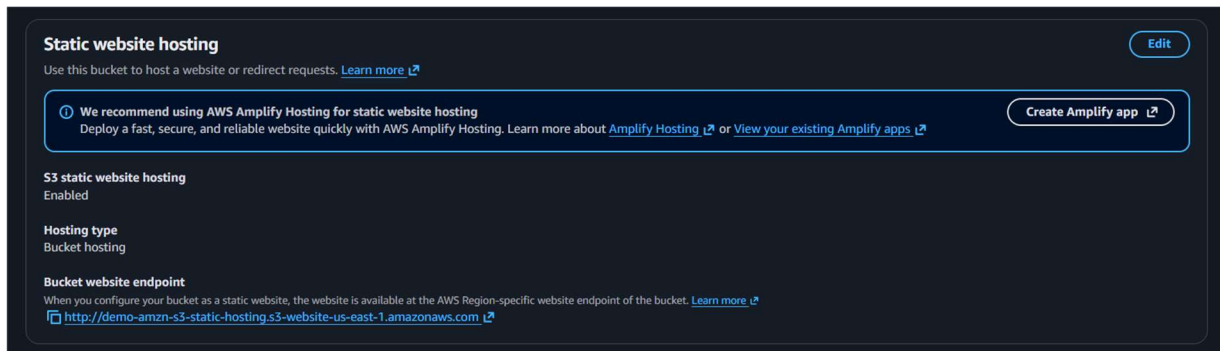


Figure 4. S3 static website hosting configuration and website endpoint.

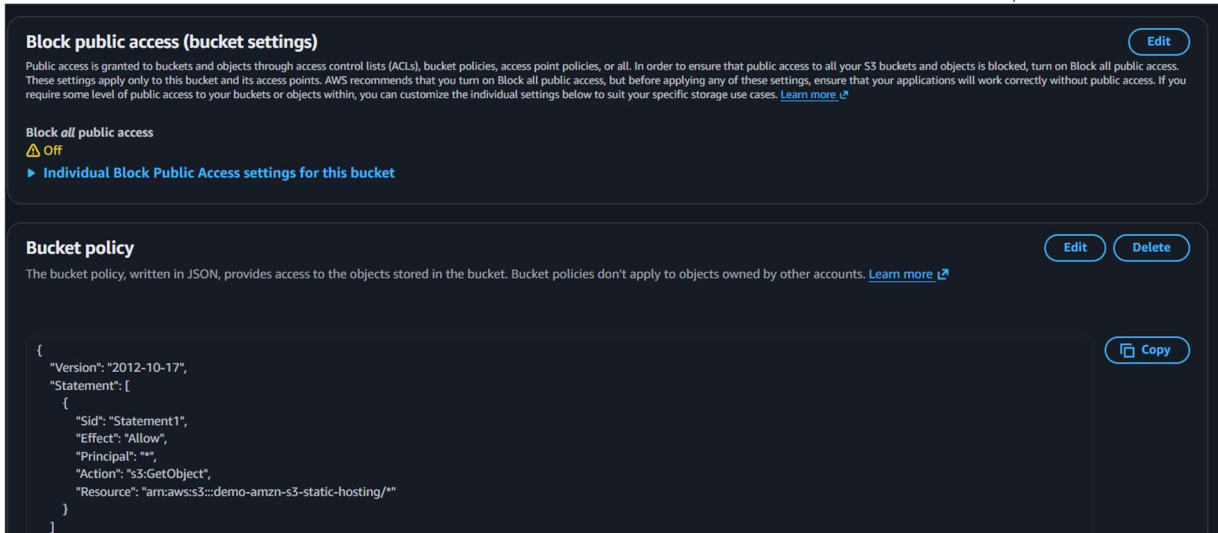


Figure 5. Bucket policy and public-access configuration used for the static website demo.

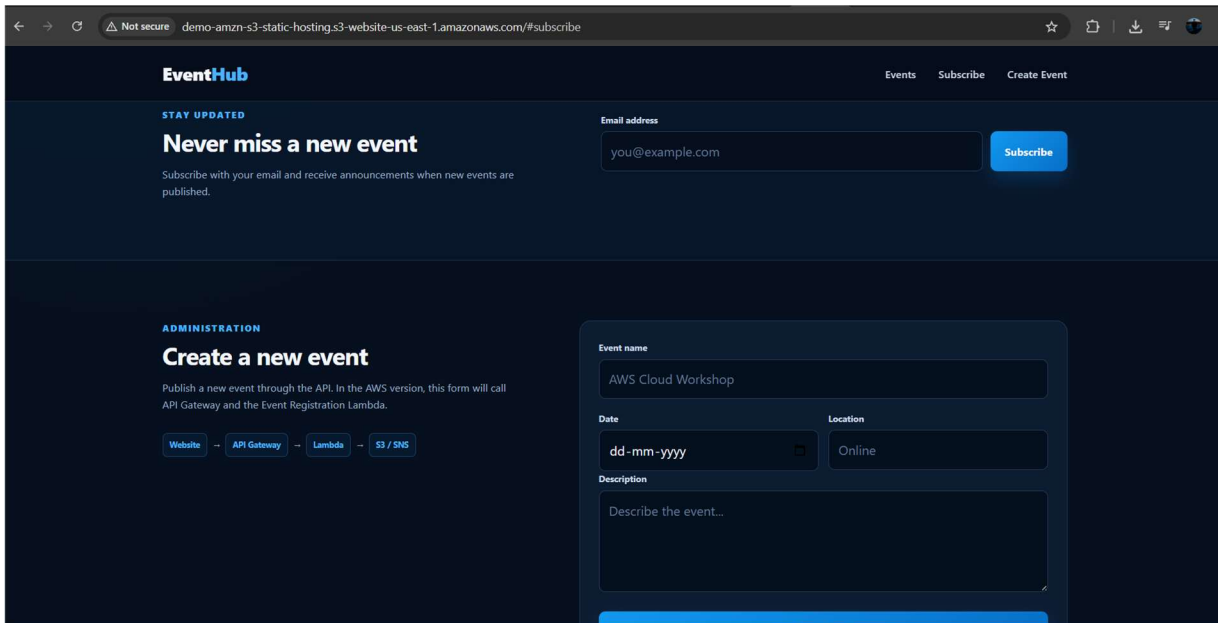


Figure 6. EventHub website running from the S3 static website endpoint.

5. Subscription Flow: Lambda + SNS

I created a Python-based Subscription Lambda from scratch. Its responsibility is to accept a subscriber email address and create an email subscription on the SNS topic.

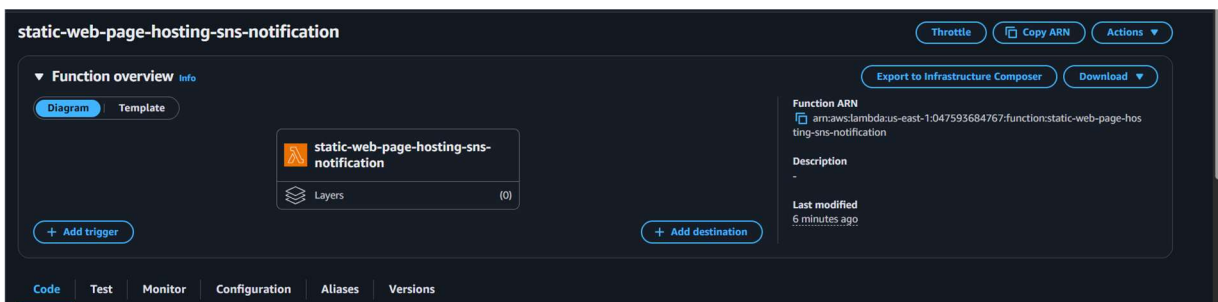


Figure 7. Subscription Lambda created from scratch using Python.

I kept the SNS topic ARN out of the source code by reading it from the Lambda environment variable `SNS_TOPIC_ARN`. This keeps the topic identifier in configuration rather than hardcoding it in the function.

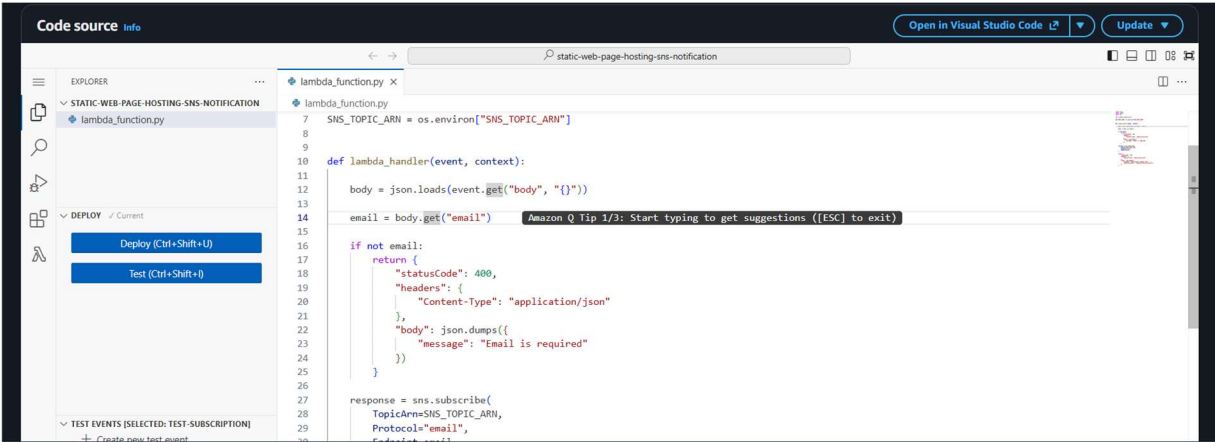


Figure 8. Python Lambda code using the SNS topic ARN from configuration.

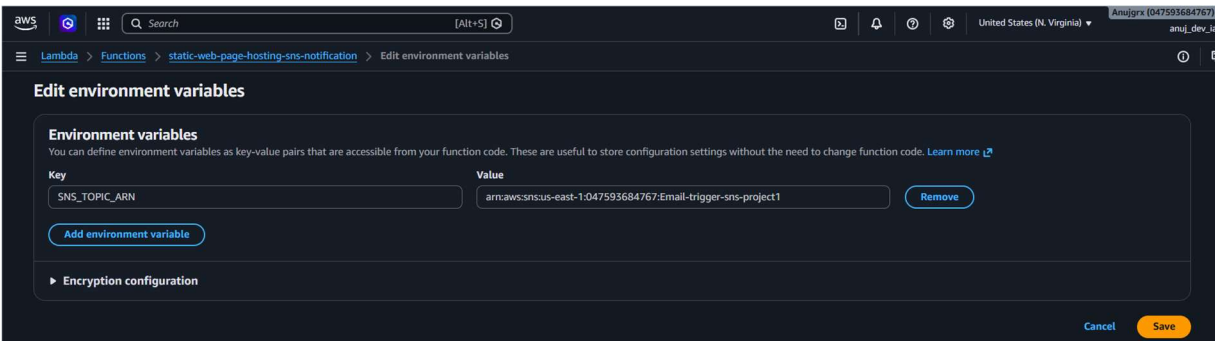


Figure 9. SNS_TOPIC_ARN configured as a Lambda environment variable.

I added the required SNS permission to the Subscription Lambda execution role so the function could create the subscription on the configured topic.

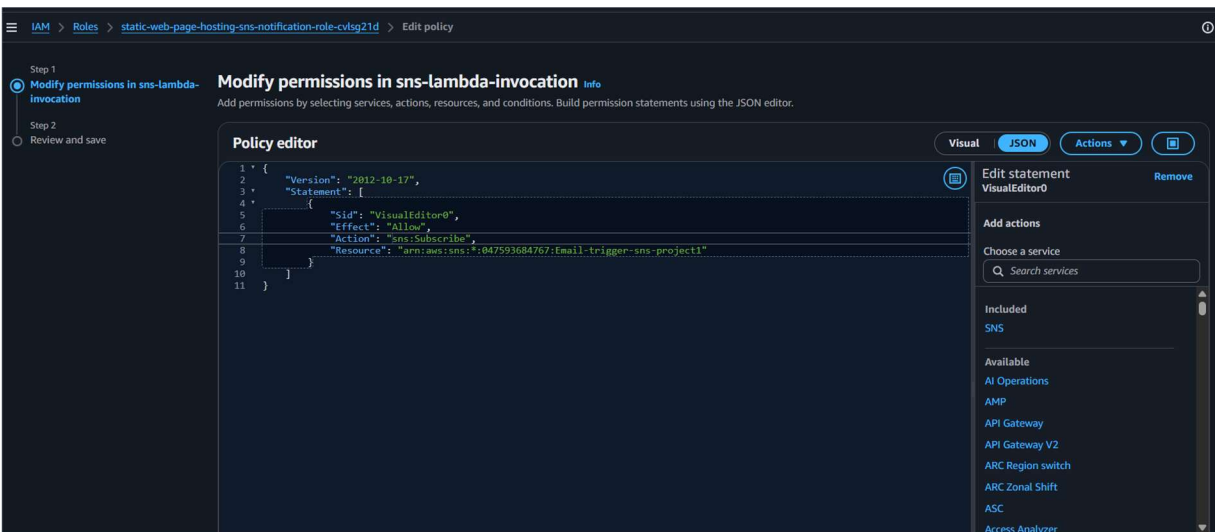


Figure 10. IAM permission assigned to the Subscription Lambda execution role.

6. API Gateway for Subscriber Registration

I created an HTTP API in Amazon API Gateway and integrated it with the Subscription Lambda. I added the POST /subscribers route so the website could submit an email address through an HTTP request.

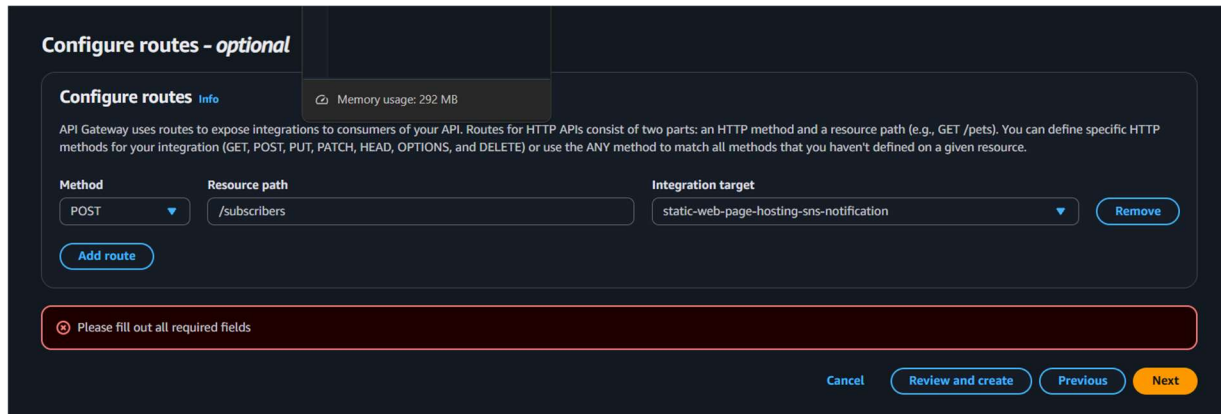


Figure 11. POST /subscribers route configured in API Gateway.

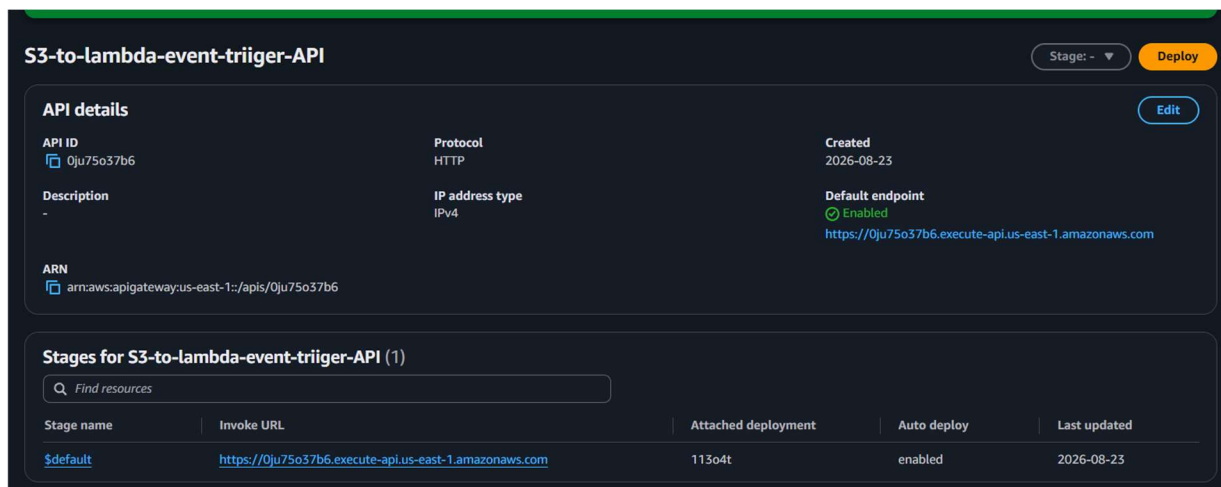


Figure 12. API Gateway HTTP API and stage information.

I tested the subscriber endpoint using PowerShell. The request returned successfully and SNS generated the subscription confirmation flow.

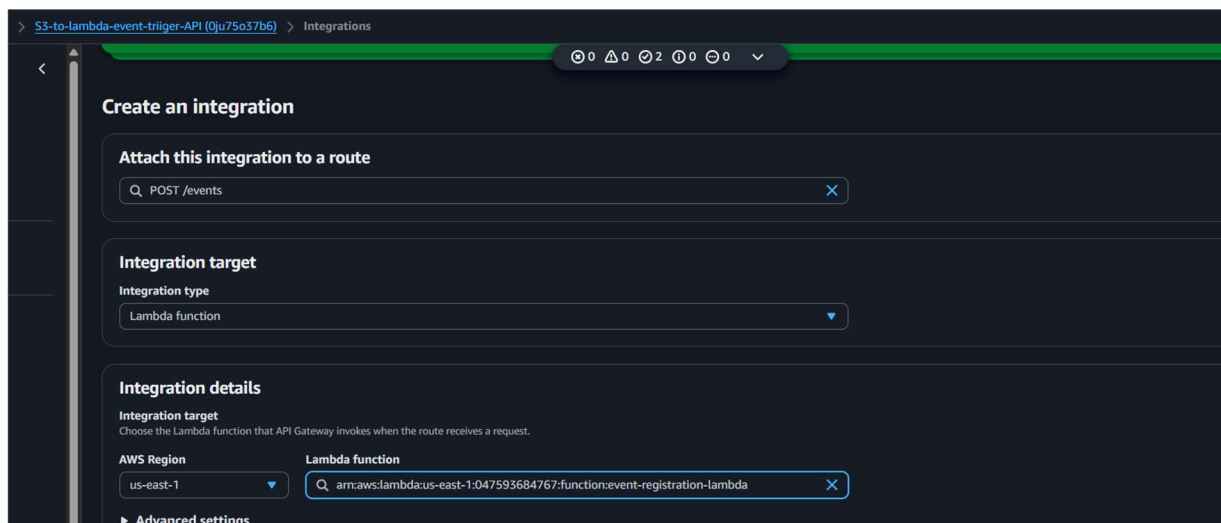


Figure 13. PowerShell test of the subscriber endpoint.

7. Connecting the S3 Website to API Gateway

I updated the website JavaScript with the API Gateway URL and configured CORS on the HTTP API so the browser could call the API from the S3 website origin. I restricted the allowed origin to the actual S3 website endpoint and allowed the POST/OPTIONS methods with the required Content-Type header.

```
const API_CONFIG = {
  subscribeEndpoint: "https://0ju75o37b6.execute-api.us-east-1.amazonaws.com/subscribers",
  createEventEndpoint: "",
  eventsEndpoint: ""
};

document.addEventListener("DOMContentLoaded", () => {

  const emailInput = document.querySelector('input[type="email"]');

  if (!emailInput) {
    console.error("Email input not found.");
    return;
  }

  const form = emailInput.closest("form");

  if (form) {
    form.addEventListener("submit", handleSubscription);
    console.log("Subscription form connected.");
    return;
  }

  const subscribeButton = [...document.querySelectorAll("button")]
    .find(button => button.textContent.trim().toLowerCase() === "subscribe");
```

Figure 14. JavaScript configuration used to call the API Gateway subscriber endpoint.

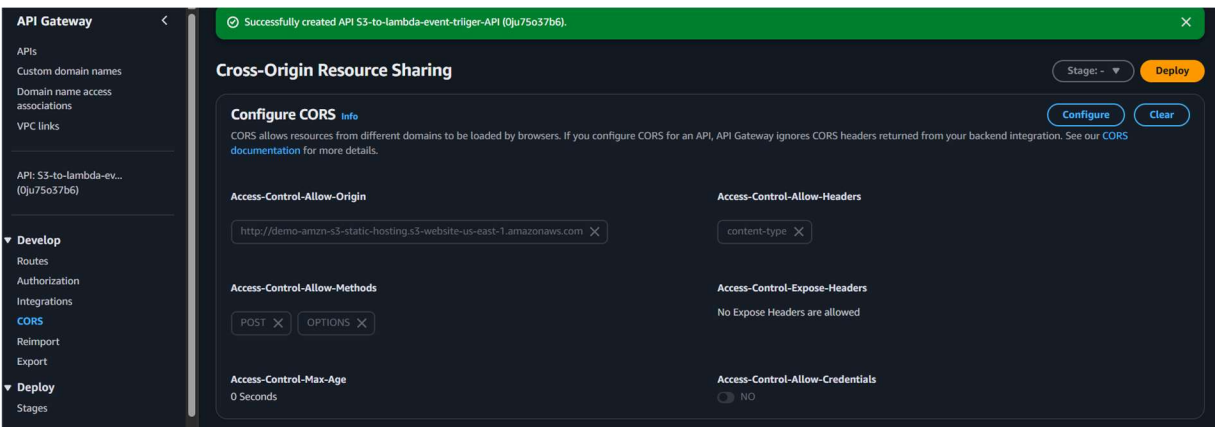


Figure 15. API Gateway CORS configuration.

After the CORS configuration was corrected to match the real S3 website origin, I tested the subscription directly from the browser and verified that the confirmation email was received.

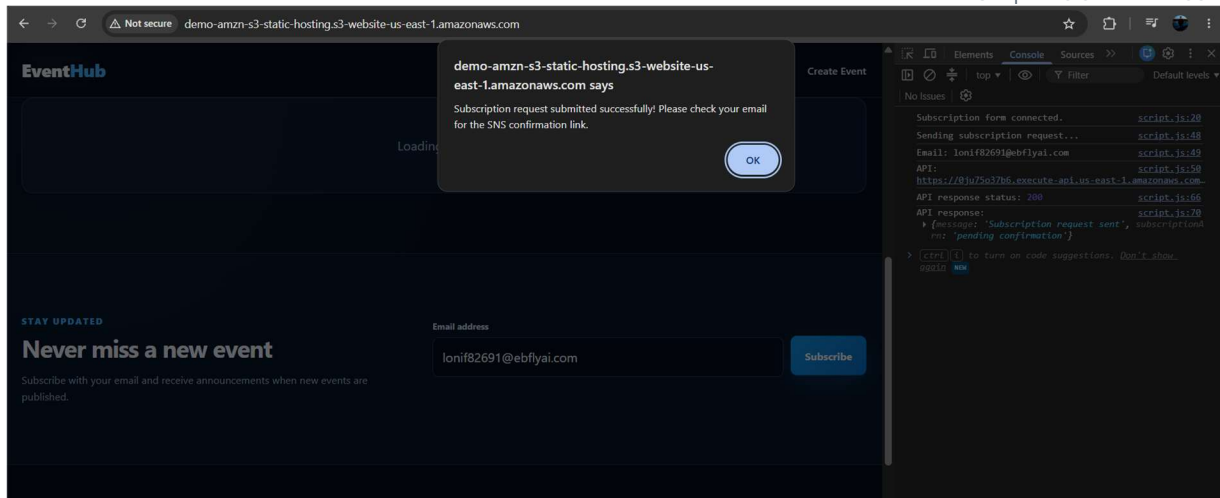


Figure 16. Browser-side subscription request and successful completion.

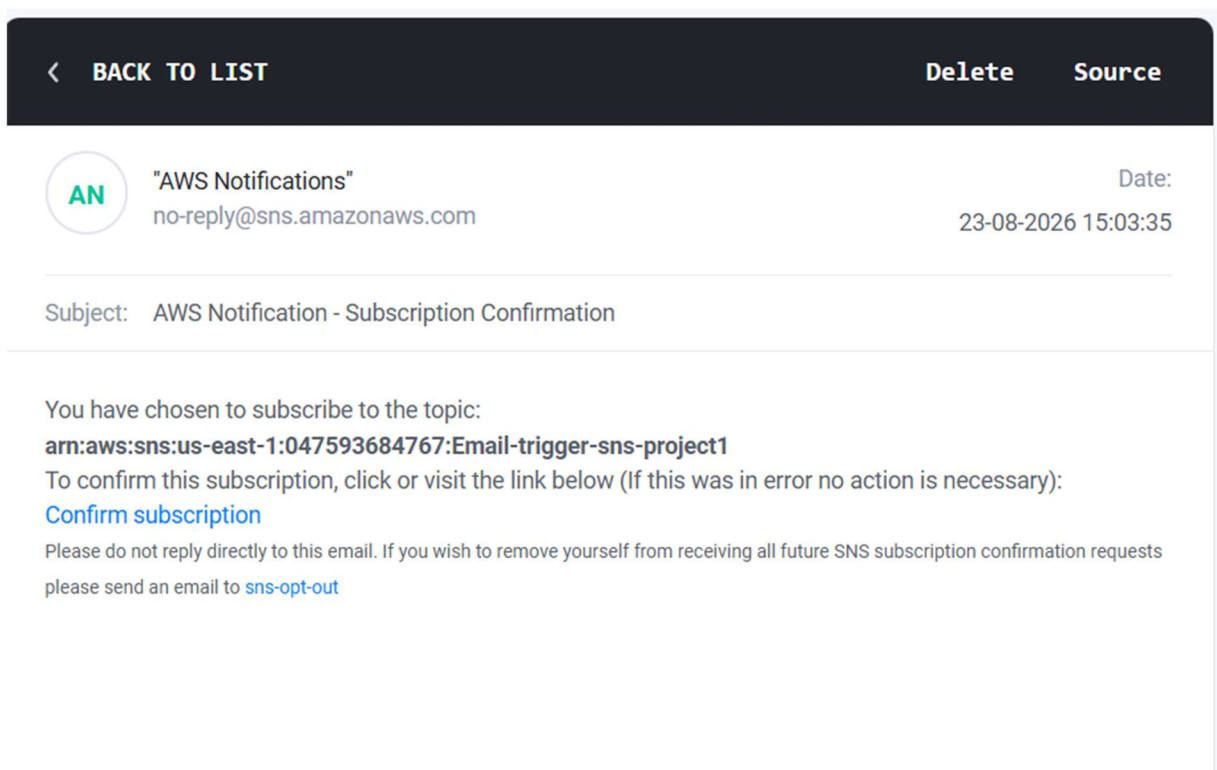


Figure 17. SNS subscription confirmation email received successfully.

8. Event Registration Flow with S3

I created a second S3 bucket dedicated to backend event data and added an events.json file to store the event records. I then created a separate Event Registration Lambda and granted it the S3 permissions required to read and update that file.

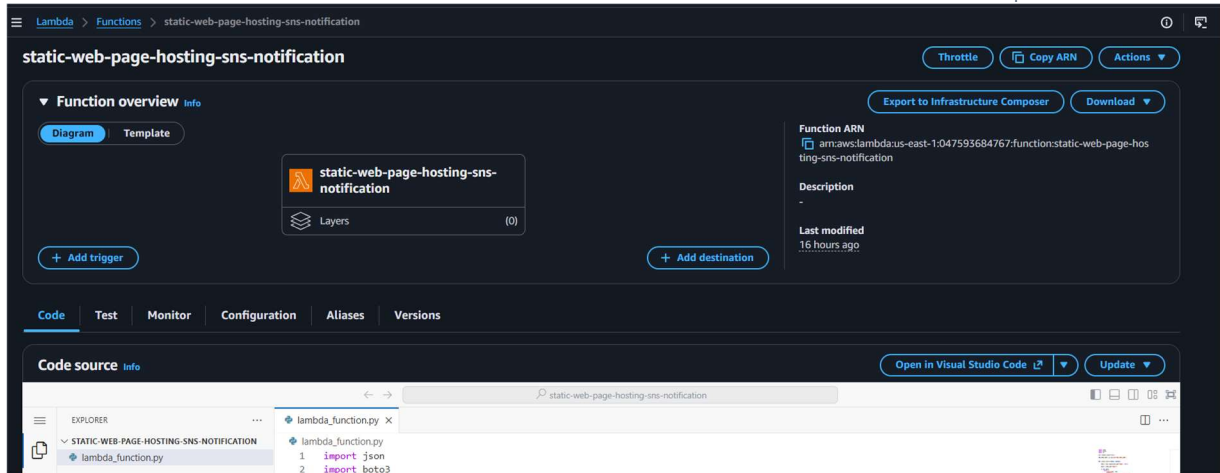


Figure 18. Event backend bucket and event data resources.

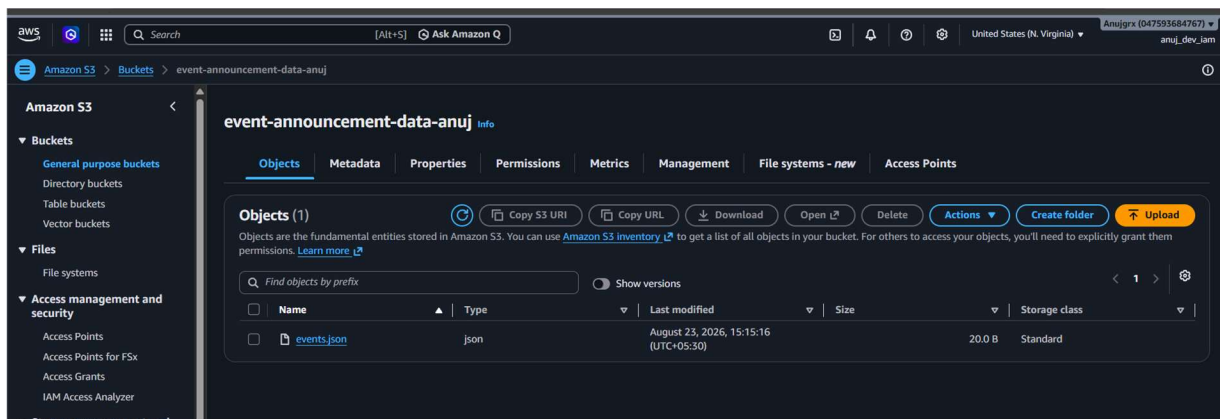


Figure 19. Event Registration Lambda implementation.

The Event Registration Lambda reads the existing JSON document, adds the new event object, and writes the updated JSON back to S3. I kept the event data in S3 to match the architecture and scope of this project.

9. Event APIs: POST and GET

I added a POST /events route in API Gateway and integrated it with the Event Registration Lambda. I then updated the frontend JavaScript so the Create Event form sends the title, date, location, and description to the API.

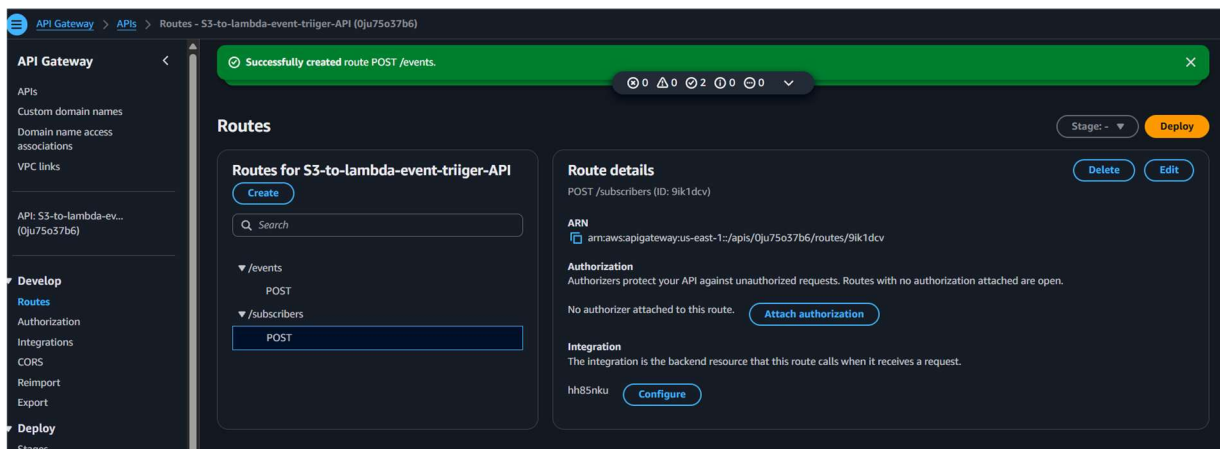


Figure 20. API Gateway route and integration for event creation.

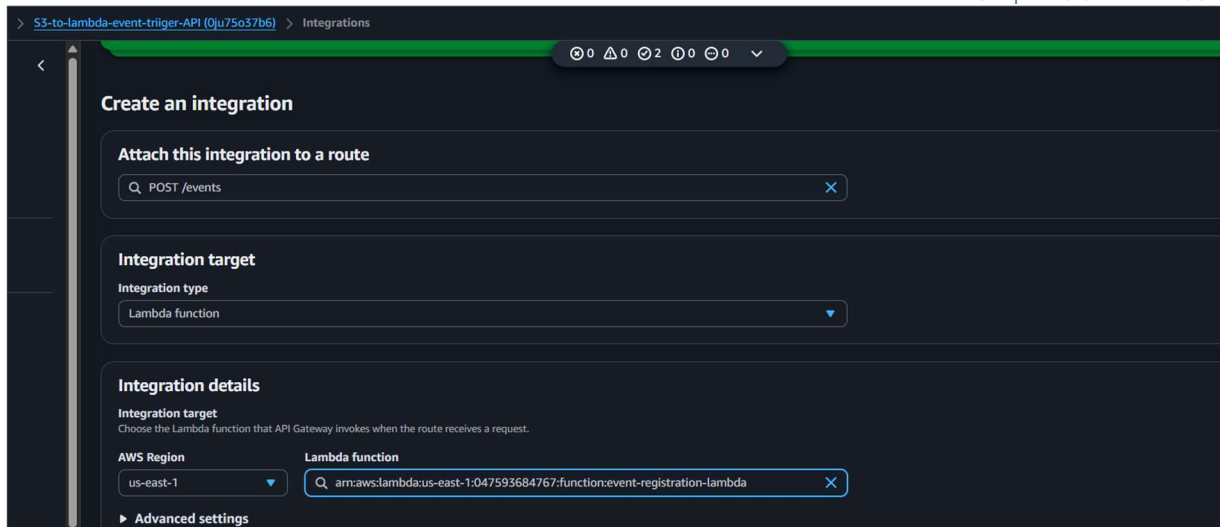


Figure 21. Lambda integration used by the POST /events route.

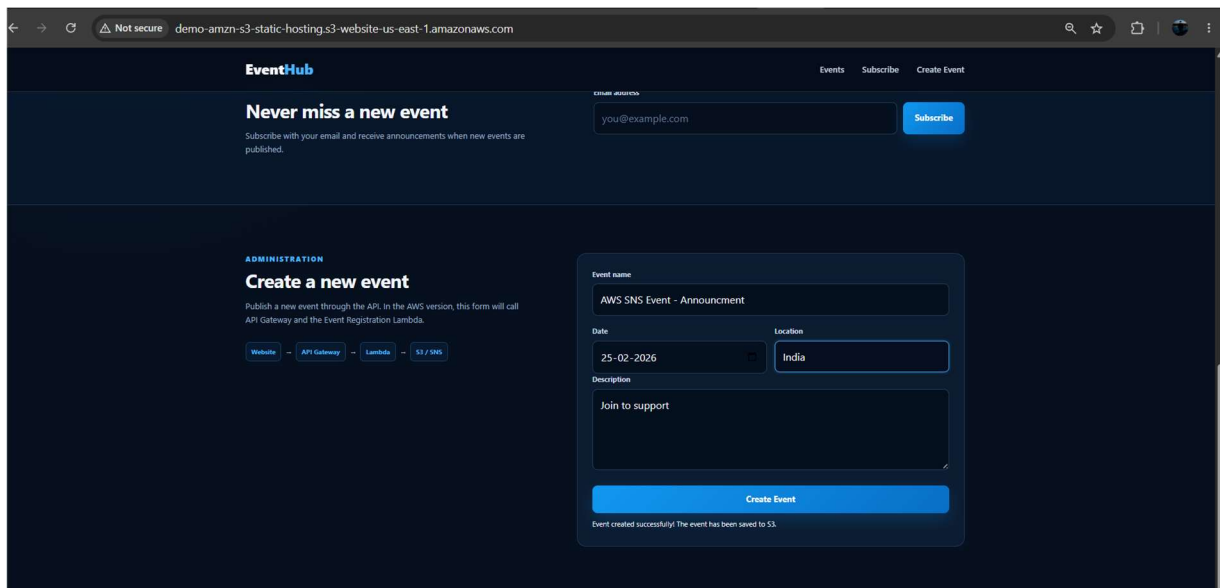
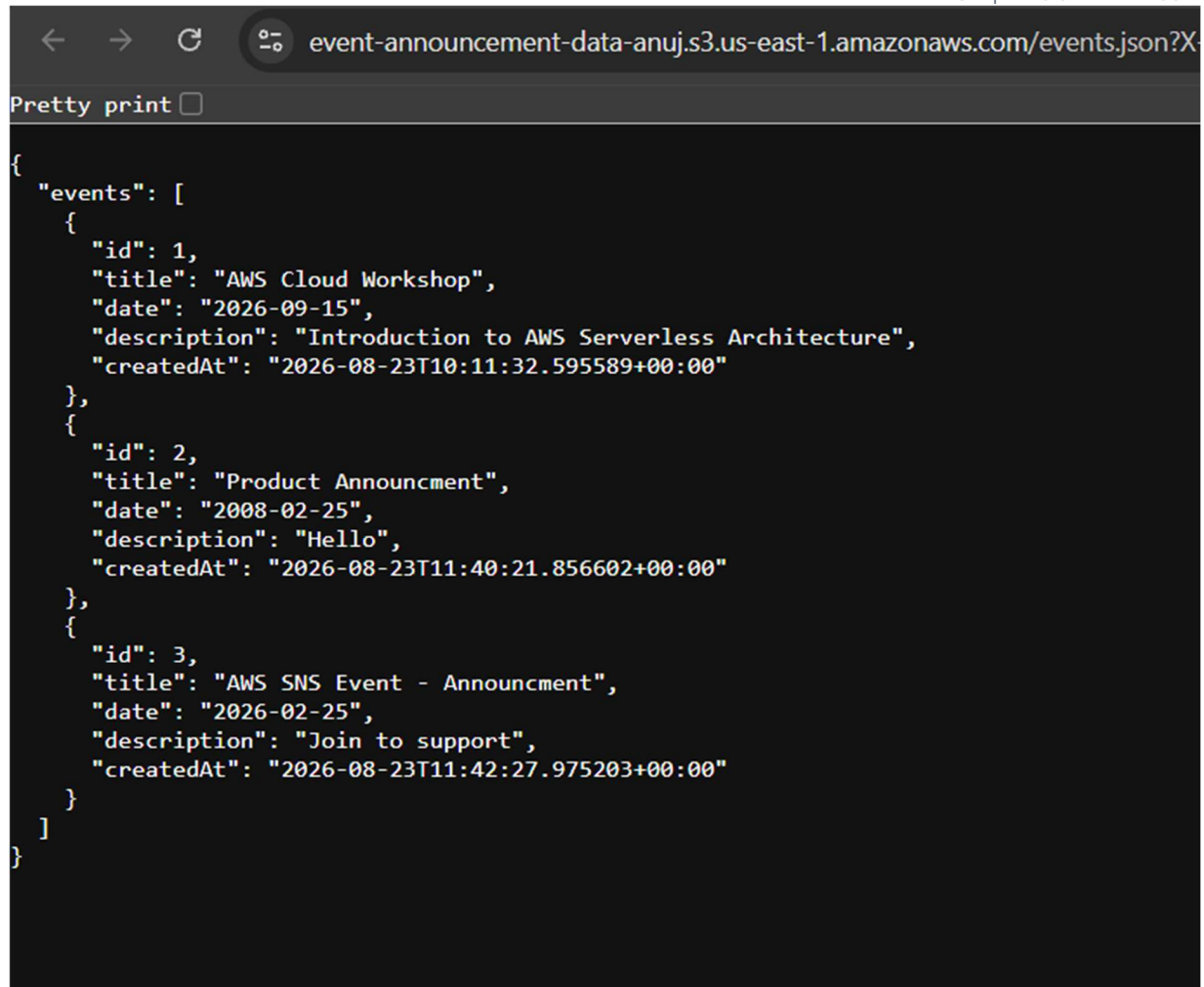


Figure 22. EventHub Create Event form connected to the backend API.

I tested the event creation flow and verified that the new records were successfully written to `events.json` in the S3 backend bucket.



```
{
  "events": [
    {
      "id": 1,
      "title": "AWS Cloud Workshop",
      "date": "2026-09-15",
      "description": "Introduction to AWS Serverless Architecture",
      "createdAt": "2026-08-23T10:11:32.595589+00:00"
    },
    {
      "id": 2,
      "title": "Product Announcement",
      "date": "2008-02-25",
      "description": "Hello",
      "createdAt": "2026-08-23T11:40:21.856602+00:00"
    },
    {
      "id": 3,
      "title": "AWS SNS Event - Announcement",
      "date": "2026-02-25",
      "description": "Join to support",
      "createdAt": "2026-08-23T11:42:27.975203+00:00"
    }
  ]
}
```

Figure 23. events.json updated successfully in the S3 backend bucket.

I then added GET /events to the same API and updated the Event Registration Lambda to support GET requests. The GET path reads events.json from S3 and returns the stored records to the website so the Upcoming Events section can be populated dynamically.

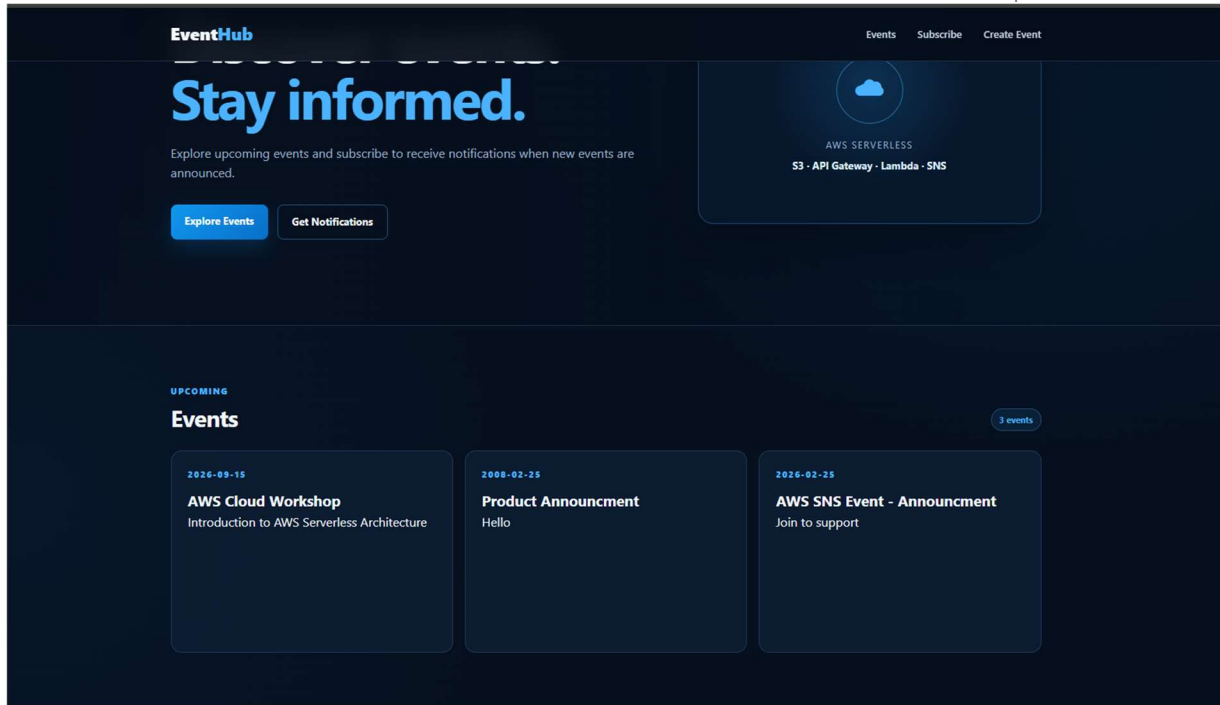


Figure 24. Final EventHub website displaying events retrieved through the GET /events flow.

10. Testing & Validation

I validated the project incrementally instead of testing the complete system only at the end. This helped me isolate configuration, IAM, API, CORS, and application-code issues while building the solution.

- I tested the Subscription Lambda and confirmed that SNS created an email subscription and sent the confirmation request.
- I tested POST /subscribers from PowerShell and then from the browser after configuring CORS.
- I created an event through the website and verified that the Event Registration Lambda updated events.json in S3.
- I tested GET /events and verified that the website displayed the stored events dynamically.
- I captured screenshots and a project walkthrough video showing the working AWS configuration and application behavior.

11. Project Outcome

The completed project demonstrates a working serverless web application in which the browser loads a static frontend from S3 and communicates with AWS-managed backend services through API Gateway. Subscriber registration is handled by Lambda and SNS, while event creation and retrieval are handled by a separate Lambda backed by an S3 JSON data file.

12. Skills Demonstrated

- Serverless application design using Amazon S3, API Gateway and AWS Lambda.
- Event notification workflow using Amazon SNS and email subscriptions.
- Python-based AWS Lambda development and API request handling.
- IAM execution-role permissions for service-to-service access.

- CORS configuration for a browser-based cross-origin API integration.
- REST-style API design using separate POST and GET operations.
- S3 object read/write operations for lightweight application data persistence.
- Incremental testing using Lambda test events, PowerShell, browser developer tools, and AWS console validation.

EventHub — a hands-on serverless AWS project built and validated end-to-end.